

Assignment-12.3

HT.NO-2203A52100

BATCH-50

Q: Lab 12: Algorithms with AI Assistance Sorting, Searching, and Algorithm Optimization Using AI Tools

Lab Objectives

The objectives of this laboratory exercise are to:

Week3 - Wednesday

- Apply AI-assisted programming techniques to implement sorting and searching algorithms.
- Analyze and compare algorithm efficiency using time and space complexity.
- Understand how AI tools can suggest optimizations and alternative algorithmic approaches.
- Strengthen problem-solving skills through real-world, data-driven scenarios.

Learning Outcomes

After completing this lab, students will be able to:

- Implement and optimize classic algorithms using AI-assisted coding tools.
- Compare multiple algorithms for the same problem and justify their selection.
- Measure and analyze runtime performance using experimental data.
- Critically review and refine AI-generated algorithmic solutions.

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records

(Name, Roll Number, CGPA).

2. Implement the following sorting algorithms using AI assistance:

o Quick Sort o

Merge Sort

3. Measure and compare runtime performance for large datasets.

4. Write a function to display the top 10 students based on CGPA.

Expected Outcome

- Correctly sorted student records.
- Performance comparison between Quick Sort and Merge Sort.
- Clear output of top-performing students.

Comments •

Task: Write a Python implementation of Bubble Sort.

- Instructions:
- Students implement Bubble Sort normally.
- Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).
- Request AI to provide time complexity analysis.
- Expected Output:
- A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

Task 3: Quick Sort and Merge Sort Comparison

- Task: Implement Quick Sort and Merge Sort using recursion.
- Instructions:
- Provide AI with partially completed functions for recursion.
- Ask AI to complete the missing logic and add docstrings.
- Compare both algorithms on random, sorted, and reverse-sorted lists.
- Expected Output:

- Working Quick Sort and Merge Sort implementations.
- AI-generated explanation of average, best, and worst-case complexities.

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory.

Task 5: Real-Time Stock Data Sorting & Searching Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

- Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
- Implement sorting algorithms to rank stocks by percentage change.
- Implement a search function that retrieves stock data instantly when a stock symbol is entered.
- Optimize sorting with Heap Sort and searching with Hash Maps.
- Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

CODE:

```
import time
import random
from typing import List, Tuple, Dict
from collections import defaultdict
import heapq
```

```
# ===== TASK 1:
```

```
Student      Records      Sorting
```

```
=====
```

```
class StudentRecord:
```

```
    """Class to store student
    information"""
    def __init__(self,
    name: str, roll_number: str, cgpa:
    float):
```

```
        self.name = name
```

```
        self.roll_number =
    roll_number
```

```
        self.cgpa = cgpa
```

```
    def __repr__(self):
```

```
        return f"Student(Name:
    {self.name}, Roll:
```

```
{self.roll_number},          CGPA:
{self.cgpa})"
```

```
def __lt__(self, other):    return
self.cgpa > other.cgpa #
For descending order
```

```
# Quick Sort Implementation for
Student      Records      def
quick_sort_students(records:
List[StudentRecord], low: int = 0,
high:      int      =      None)  ->
List[StudentRecord]:
```

```
    """
    Quick Sort algorithm for sorting
    student records by CGPA in
    descending order.
```

```
    Time Complexity:  $O(n \log n)$ 
    average,  $O(n^2)$  worst case    Space
    Complexity:  $O(\log n)$  due to
    recursion
```

```
    Args:
        records: List of StudentRecord
    objects
```

low: Start index

high: End index

Returns:

Sorted list of student records

""" if high is None:

high = len(records) - 1

if low < high:

Partition and get pivot index

pi =

_partition_students(records, low,
high)

Recursively sort left and right

parts

quick_sort_students(records, low,

pi - 1)

quick_sort_students(records, pi +
1, high)

return records

def

_partition_students(record

s: List[StudentRecord], low: int,

high: int) -> int:

```
"""Helper function to partition
the list"""
```

```
    pivot = records[high].cgpa
```

```
    i = low - 1
```

```
    for j in range(low, high):
        if records[j].cgpa > pivot:
```

```
            # Descending order
```

```
                i += 1
```

```
            records[i], records[j] =
records[j], records[i]
```

```
    records[i + 1], records[high] =
records[high], records[i + 1]
    return i + 1
```

```
# Merge Sort Implementation for
```

```
Student Records def
```

```
merge_sort_students(records:
```

```
List[StudentRecord]) ->
```

```
List[StudentRecord]:
```

```
    """
```

```
    Merge Sort algorithm for sorting
student records by CGPA in
descending order.
```

Time Complexity: $O(n \log n)$ in all cases

Space Complexity: $O(n)$ for temporary arrays

Args:

records: List of StudentRecord objects

Returns:

Sorted list of student records

```
""" if
len(records) <= 1:
    return records

    mid = len(records) // 2
    left =
    merge_sort_students(records[:mid])

    right =
    merge_sort_students(records[mid:])

    return _merge_students(left,
right)

def _merge_students(left:
List[StudentRecord], right:
```

List[StudentRecord]) ->

List[StudentRecord]: """Helper
function to merge two sorted
lists"""

result = []
i

= j = 0

while i < len(left) and j <

len(right):

if left[i].cgpa >=

right[j].cgpa: # Descending order

result.append(left[i]) i +=

1 else:

result.append(right[j])

j += 1

result.extend(left[i:])

result.extend(right[j:]) return

result

def

display_top_10_students(records:

List[StudentRecord]) -> None:

"""Display top 10 students by
CGPA"""

print("\n" + "="*60)

```

print("TOP 10 STUDENTS BY
CGPA")

print("="*60)

print(f"{'Rank':<6}{'Name':<20}{'
Roll Number':<15}{'CGPA':<10}")

print("-"*60)

```

```

        for i, student in
enumerate(records[:10], 1):
print(f"{'i':<6}{student.name:<
20}{student.roll_number:<15}{stu
dent.cgpa:<10.2f}")
print("="*60)

```

```

# ===== TASK 2:
Bubble Sort with Comments
=====

```

```

def bubble_sort(arr: List[float]) ->
List[float]:
    """
    Bubble Sort Algorithm - Simplest
    sorting algorithm

```

```

    Time Complexity Analysis:    -
    Best Case: O(n) - when array is
    already sorted    - Average Case:

```

$O(n^2)$ - random distribution -

Worst Case: $O(n^2)$ - when array is
reverse sorted

Space Complexity: $O(1)$ - sorts
inplace

Args:

arr: List of numbers to be
sorted

Returns:

Sorted list in ascending order

```
"""    n =  
len(arr)
```

```
# Outer loop: controls number of  
passes
```

```
# Each pass moves the largest  
unsorted element to its correct  
position for i in range(n): #  
Flag to optimize: if no swaps occur,  
array is sorted swapped = False
```

```
# Inner loop: compares  
adjacent elements
```

```
# i elements at the end are
already in correct position, so we
reduce range
```

```
for j in range(0, n - i - 1):
```

```
# Comparison: swap if
current element is greater than
next element
```

```
if arr[j] > arr[j + 1]:
```

```
# Swap operation:
```

```
exchange positions of two
elements
```

```
arr[j], arr[j + 1] = arr[j + 1],
arr[j] swapped = True
```

```
# Optimization: if no swaps
occurred in a pass, array is sorted
# Termination condition: exit early
if array becomes sorted if not
swapped: break
```

```
return arr
```

```
# ===== TASK 3:
Quick Sort vs Merge Sort
=====
```

```
def quick_sort_generic(arr:
```

List[float], low: int = 0, high: int =

None) -> List[float]:

"""

Quick Sort Implementation using
Recursion

Complexity Analysis: - Best
Case: $O(n \log n)$ - balanced
partition

- Average Case: $O(n \log n)$ -
Worst Case: $O(n^2)$ - pivot is always
smallest/largest

Space Complexity: $O(\log n)$ -
recursion stack

Args:

arr: List to sort

low: Starting index

high: Ending index

Returns:

Sorted list """

if high is None:

high = len(arr) - 1

if low < high:

```
    pi = _partition_generic(arr,
low, high)
```

```
    quick_sort_generic(arr, low, pi
- 1)    quick_sort_generic(arr, pi
+ 1, high)
```

```
    return arr
```

```
def    _partition_generic(arr:
List[float], low: int, high: int) -> int:
```

```
    """
```

```
    Partition helper for Quick Sort
    Selects pivot and partitions array
    into smaller and larger elements
```

```
    """    pivot =
arr[high]    i = low
- 1
```

```
    for j in range(low, high):
```

```
    if arr[j] < pivot:
```

```
        i += 1        arr[i], arr[j]
= arr[j], arr[i]
```

```
    arr[i + 1], arr[high] = arr[high],
arr[i + 1]    return i + 1
```

```
def    merge_sort_generic(arr:
```

List[float]) -> List[float]:

"""

Merge Sort Implementation
using Recursion

Complexity Analysis:

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$

Space Complexity: $O(n)$ -
requires temporary arrays for
merging

Args: arr: List
to sort

Returns:

Sorted list

""" if len(arr)

≤ 1 : return

arr

mid = len(arr) // 2

left =

merge_sort_generic(arr[:mid])

right =

```
merge_sort_generic(arr[mid:])
return _merge_generic(left,
right)
```

```
def      _merge_generic(left:
List[float], right: List[float]) ->
List[float]:
```

```
    """
```

```
    Merge helper for Merge Sort
    Combines two sorted arrays into
    one sorted array
```

```
    """
```

```
    result = []
```

```
    i = j = 0
```

```
    while i < len(left) and j <
len(right):    if left[i] <=
right[j]:
```

```
    result.append(left[i])
```

```
    i += 1    else:
```

```
        result.append(right[j])
```

```
        j += 1
```

```
    result.extend(left[i:])
```

```
    result.extend(right[j:])    return
```

```
    result
```

```
# ===== TASK 4:
```

```
Inventory Management System
```

```
=====
```

```
class Product:    """Product class  
for inventory management"""
```

```
    def __init__(self, product_id: int,  
name: str, price: float, quantity:  
int):
```

```
        self.product_id = product_id  
self.name = name    self.price  
= price    self.quantity =  
quantity
```

```
    def __repr__(self):  
        return f"Product(ID:  
{self.product_id},    Name:  
{self.name},    Price: ${self.price},  
Qty: {self.quantity})"
```

```
class InventorySystem:
```

```
    """Inventory  
    Management System with  
optimized search and sort"""
```

```
    def __init__(self):
```

```

        self.products: List[Product] = []
self.id_index: Dict[int, Product] = {}
# Hash map for O(1) ID lookup
self.name_index: Dict[str,
List[Product]] = defaultdict(list) #
For name search

```

```

def add_product(self, product:
Product):
    """Add product to
inventory"""
self.products.append(product)
self.id_index[product.product_id]
= product
self.name_index[product.name
e.lower()].append(product)

```

```

def search_by_id(self,
product_id: int) -> Product:
    """
Search product by ID - O(1)
using hash map
Recommended: Hash Map for
exact ID match
    """
    return
self.id_index.get(product_id,

```

None)

```
def search_by_name(self, name:  
str) -> List[Product]:
```

```
    """
```

```
        Search product by name -  $O(1)$   
average using hash map  
Recommended: Hash Map for  
name lookups
```

```
    """
```

```
        return  
self.name_index.get(name.lower()  
, [])
```

```
def sort_by_price(self) ->  
List[Product]:
```

```
    """
```

```
        Sort products by price -  $O(n \log n)$   
using Merge Sort  
Recommended: Merge Sort for  
consistent  $O(n \log n)$ 
```

```
    """
```

```
        return  
merge_sort_generic([p.price for p  
in self.products])
```

```
def sort_by_quantity(self) ->  
List[Product]:
```

```

        """
        Sort products by quantity - O(n
log n)

        Recommended: Quick Sort for
average case efficiency
        """

```

```

            return
        quick_sort_generic([p.quantity for
p in self.products])

```

```

# ===== TASK 5:
Stock      Data      Analysis
=====

```

```

class StockData:
    """Class to store stock
information"""
    def __init__(self,
symbol: str,
opening_price: float,
closing_price: float):
        self.symbol = symbol
        self.opening_price =
opening_price
        self.closing_price =
closing_price
        self.daily_change =

```

```
closing_price - opening_price
```

```
self.percentage_change =
```

```
(self.daily_change /  
opening_price) * 100  
if
```

```
opening_price != 0 else 0
```

```
def __repr__(self):
```

```
    return f"Stock({self.symbol},
```

```
Open: ${self.opening_price:.2f},
```

```
Close: ${self.closing_price:.2f},
```

```
Change:
```

```
{self.percentage_change:.2f}%")
```

```
def generate_stock_data(count:
```

```
int = 100) -> List[StockData]:
```

```
    """
```

```
    Simulate stock price data for  
testing
```

```
    Args:
```

```
        count: Number of stocks to  
generate
```

```
    Returns:
```

```
        List of StockData objects
```

```
    """
```

```

symbols = ['AAPL', 'GOOGL',
'MSFT', 'AMZN', 'TSLA', 'META',
'NVDA', 'JPM', 'BA', 'WMT']

```

```

stocks = []

```

```

for i in range(count):

    symbol =
random.choice(symbols) + str(i %
10)

    opening =
round(random.uniform(50, 500),
2)    closing = round(opening *
random.uniform(0.95, 1.05), 2)
stocks.append(StockData(sym bol,
opening, closing))

```

```

return stocks

```

```

class StockAnalysisSystem:
    """Stock data analysis system with
    optimized search and sort"""

```

```

    def __init__(self, stocks:
List[StockData]):    self.stocks =
stocks    self.symbol_map:
Dict[str, StockData] =
{stock.symbol: stock for stock in
stocks}

```

```

def
    search_by_symbol(self,
symbol: str) -> StockData:
    """
        Search stock by symbol using
Hash      Map      -      O(1)
Recommended: Hash Map for
instant lookup
    """

        return
self.symbol_map.get(symbol,
None)

def
sort_by_percentage_gain(self) ->
List[StockData]:
    """
        Sort stocks by percentage
change using Heap Sort - O(n log n)
Recommended: Heap Sort for
consistent performance
    """

        return sorted(self.stocks,
key=lambda          x:
x.percentage_change,
reverse=True)

```

```

def
sort_by_gain_heap_sort(self) ->
List[StockData]:
    """
        Heap Sort implementation for
stock sorting

        Time Complexity:  $O(n \log n)$ 
guaranteed

        Space Complexity:  $O(1)$  if not
counting output
    """
    heap = []
    for stock in self.stocks:
        # negate for max heap
behavior
        heapq.heappush(heap,
(stock.percentage_change,
stock.symbol, stock))

    result = []
    while heap:
        _, _, stock =
heapq.heappop(heap)
        result.append(stock)

    return result

```

```

def get_top_gainers(self, n: int =
10) -> List[StockData]:
    """Get top n stocks by
percentage gain"""

    return sorted(self.stocks,
key=lambda x:
x.percentage_change,
reverse=True)[:n]

```

```

def get_top_losers(self, n: int =
10) -> List[StockData]:
    """Get top n stocks by
percentage loss"""
    return
sorted(self.stocks,
key=lambda x:
x.percentage_change)[:n]

```

```

# =====
PERFORMANCE TESTING
=====

```

```

def
measure_sorting_performance(da
ta: List[float], algorithm_name: str,
algorithm_func) -> Tuple[float,
List[float]]:

```

```
"""
Measure sorting algorithm
performance
```

Returns:

 Tuple of (execution_time,
sorted_data)

```
"""
start_time = time.time()

    sorted_data =
algorithm_func(data.copy())
end_time = time.time()

return end_time - start_time,
sorted_data
```

```
def performance_comparison():
    """Compare Quick Sort and
Merge Sort on different
datasets"""
    sizes = [100,
500, 1000]
```

```
print("\n" + "="*70)

    print("PERFORMANCE
COMPARISON: QUICK SORT vs
MERGE SORT")

print("="*70)
```

```

        print(f'{"Dataset Size":<20}{"Quick Sort (sec)":<20}{"Merge Sort (sec)":<20}{"Winner":<10}')}')
    print("-"*70)

```

```

    for size in sizes:        # Random
data    data = [random.uniform(1,
1000) for _ in range(size)]

```

```

        qs_time, _ =
measure_sorting_performance(da
ta,        "Quick Sort",
quick_sort_generic)

```

```

        ms_time, _ =
measure_sorting_performance(da
ta,        "Merge Sort",
merge_sort_generic)

```

```

        winner = "Quick Sort" if
qs_time < ms_time else "Merge
Sort"

```

```

    print(f'{"size":<20}{"qs_time":<2
0.6f}{"ms_time":<20.6f}{"winner":<10
}')

```

```

    print("-"*70)

```

```

# ===== MAIN

EXECUTION

=====

if __name__ == "__main__":
    print("\n" + "="*70)

    print("LAB 12: SORTING,
SEARCHING, AND ALGORITHM
OPTIMIZATION")

    print("="*70)

    # TASK 1: Student Records
    print("\n--- TASK 1: Student
Records Sorting ---")    students = [

        StudentRecord("Akhir
Ahmed", "001", 9.2),

        StudentRecord("Priya Singh",
"002", 8.9),

        StudentRecord("Raj Patel",
"003", 9.5),

        StudentRecord("Sara Khan",
"004", 8.7),

        StudentRecord("Amit Kumar",
"005", 9.1),

        StudentRecord("Lisa Wong",
"006", 9.3),

        StudentRecord("John Smith",

```

```

"007", 8.5),
    StudentRecord("Emma Davis",
"008", 9.4),
    StudentRecord("Rahul
Verma", "009", 8.8),
    StudentRecord("Zoe Chen",
"010", 9.6),
]

```

```

# Quick Sort

```

```

students_qs =
[StudentRecord(s.name,
s.roll_number, s.cgpa) for s in
students] qs_start = time.time()
quick_sort_students(students_q
s) qs_time = time.time() -
qs_start

```

```

# Merge Sort

```

```

students_ms =
[StudentRecord(s.name,
s.roll_number, s.cgpa) for s in
students] ms_start = time.time()
merge_sort_students(students_
ms) ms_time = time.time() -
ms_start

```

```

print(f"Quick Sort Time:

```

```
{qs_time:.6f} seconds")
```

```
    print(f"Merge Sort
```

```
    Time:
```

```
{ms_time:.6f} seconds")
```

```
display_top_10_students(stude  
nts_qs)
```

```
# TASK 2: Bubble Sort  print("\n-  
-- TASK 2: Bubble Sort with  
Comments ---")  test_array = [64,  
34, 25, 12, 22,  
11, 90]
```

```
    print(f"Original    Array:  
{test_array}")
```

```
        sorted_array    =  
bubble_sort(test_array.copy())
```

```
    print(f"Sorted    Array:  
{sorted_array}")    print("Time  
Complexity:  $O(n^2)$  average and  
worst case,  $O(n)$  best case")  
print("Space Complexity:  $O(1)$ ")
```

```
# TASK 3: Quick vs Merge Sort  
Comparison
```

```
    print("\n--- TASK 3: Quick Sort vs  
Merge Sort Comparison ---")  
performance_comparison()
```

```

# TASK 4: Inventory
Management

print("\n--- TASK 4: Inventory
Management System ---")

inventory = InventorySystem()

inventory_products = [
    Product(1, "Laptop", 999.99,
50),
    Product(2, "Mouse", 29.99,
200),
    Product(3, "Keyboard", 79.99,
150),
    Product(4, "Monitor", 299.99,
75),
    Product(5, "Headphones",
149.99, 100),
]

for product in
inventory_products:
    inventory.add_product(produ
ct)

print("Inventory Management -
Algorithm Recommendations:")

```

```

print("="*60)
print("Operation\t\tRecommended Algorithm\tJustification")
print("-"*60)

print("Search by ID\t\tHash Map\t\tO(1) lookup")
print("Search by Name\t\tHash Map\t\tO(1) average lookup")
print("Sort by Price\t\tMerge Sort\t\tO(n log n) guaranteed")
print("Sort by Quantity\tQuick Sort\t\tO(n log n) average")
print("="*60)

```

```

# TASK 5: Stock Analysis
print("\n--- TASK 5: Stock Data Sorting & Searching ---")

```

```

stocks = generate_stock_data(20)
stock_system = StockAnalysisSystem(stocks)

```

```

print("\nTop 5 Gainers:")
print("-"*70)

for i, stock in

```

```

enumerate(stock_system.get_top
_gainers(5), 1):      print(f"{i}.
{stock.symbol}:
{stock.percentage_change:+.2f}%"
)

```

```

print("\nTop 5 Losers:") print("-
"*70)

```

```

        for i, stock in
enumerate(stock_system.get_top
_losers(5), 1):      print(f"{i}.
{stock.symbol}:
{stock.percentage_change:+.2f}%"
)

```

Search example

```

        search_symbol =
stocks[0].symbol

        found_stock =
stock_system.search_by_symbol(s
earch_symbol) print(f"\nSearch
Result for
{search_symbol}: {found_stock}")

```

```

        print("\n" + "="*70)
print("="*70)

```

OUTPUT:

```
PS C:\Users\krish\OneDrive\Documents\AIT\Py_files> & c:\Users\krish\OneDrive\Documents\AIT\Py_files\venv\scripts\python.exe "c:\Users\krish\OneDrive\Documents\AIT\Py_files\12.1 (1).py"

=====
LAB 12: SORTING, SEARCHING, AND ALGORITHM OPTIMIZATION
=====

--- TASK 1: Student Records Sorting ---
Quick Sort Time: 0.000003 seconds
Merge Sort Time: 0.000042 seconds

=====
TOP 10 STUDENTS BY CGPA
=====
Rank  Name      Roll Number  CGPA
-----
1.   Zoe Chen      010        9.40
2.   Raj Patel     003        9.50
3.   Emma Davis    008        9.40
4.   Lisa Wong     006        9.30
5.   Amir Ahmed   001        9.20
6.   Aali Kumar    005        9.10
7.   Priya Singh   002        8.90
8.   Rahul Verma   009        8.80
9.   Sara Khan     004        8.70
10.  John Smith    007        8.50
=====

--- TASK 2: Bubble Sort with Comments ---
Original Array: [64, 34, 25, 12, 22, 11, 90]
Sorted Array: [11, 12, 22, 25, 34, 64, 90]
Time Complexity: O(n^2) average and worst case, O(n) best case
Space Complexity: O(1)

--- TASK 3: Quick Sort vs Merge Sort Comparison ---

=====
PERFORMANCE COMPARISON: QUICK SORT vs MERGE SORT
=====
Dataset Size    Quick Sort (sec)  Merge Sort (sec)  Winner
-----
100              0.000142          0.000357          Quick Sort
500              0.001125          0.001748          Quick Sort
1000             0.003054          0.009814          Quick Sort
=====

--- TASK 4: Inventory Management System ---
Inventory Management - Algorithm Recommendations:
=====
Operation      Recommended Algorithm  Justification
-----
Search by ID    Hash Map               O(1) Lookup
Search by Name  Hash Map               O(1) average Lookup
Sort by Price   Merge Sort             O(n log n) guaranteed
Sort by Quantity Quick Sort             O(n log n) average
=====
```

```
--- TASK 5: Stock Data Sorting & Searching ---

Top 5 Gainers:
-----
1. MSFTB: +4.45%
2. TSLA2: +3.25%
3. BABA: +2.83%
4. MSFT3: +1.76%
5. NVDA4: +1.61%

Top 5 Losers:
-----
1. JPM5: -4.85%
2. BAF: -4.73%
3. BAS: -3.84%
4. META4: -3.84%
5. TSLAB: -2.78%

Search Result for NVDA0: Stock(NVDA0, Open: $437.63, Close: $443.66, Change: 1.38%)

=====
PS C:\Users\krish\OneDrive\Documents\AIT\Py_files> |
```


